CAST: 리전 용량 제약을 고려한 탄소 인지 시공간 워크로드 스케줄링

CAST: Carbon-Aware Spatio-Temporal Scheduling under Capacity Constraints

초록 데이터센터 탄소 배출량은 실행 리전과 시각에 따라 크게 달라진다. 선행 연구는 대부분 리전 용량을 무제한으로 두어 최적해가 가장 깨끗한 리전으로 쏠린다. 본 실험에서도 작업의 99.75%가 프랑스로 몰려 실행이 불가능하다. 본 연구는 LSTM 탄소집약도 예측 위에서 정수 계획법으로 리전을 정하고, 마감 안에서 실행 시각을 옮기되 리전별 용량 상한을 매 슬롯 강제하는 CAST를 제안한다. 탄소-지연 가중치는 슬롯마다 파레토 무릎점에서 자동 결정한다. 8개 리전 1년치 실측 데이터와 합성 작업 146,000건 실험에서 마감 위반 없이 배출량을 63.03% 줄였다. 용량을 무시하면 65.93%까지 줄지만 상한 위반은 227건에서 20,208건으로 늘어난다. 반대로 용량을 하드 제약으로 두면 상한 위반은 사라지는 대신 마감 위반이 3,044건 생긴다 — 둘을 함께 지키는 배치는 없다. 핵심은 총계가 아닌 분포다. 용량이 없을 때 시간 이동 절감의 71.9%가 캘리포니아에서 나오고, 그 초과 시간의 87.4%가 현지 한낮인 UTC 19~21시에 몰린다. 시간 이동이 가치를 만드는 리전과 용량이 이를 막는 리전이 같다.

주제어: 탄소 인지 스케줄링, 워크로드 이동, 정수 계획법, 리전 용량, LSTM

Abstract The carbon footprint of a cloud workload varies with where and when it runs. Prior work has mostly assumed unlimited regional capacity, under which the optimum collapses onto the cleanest region: on our data, 99.75% of jobs go to France, a placement that cannot be executed. We present CAST, which forecasts carbon intensity with an LSTM, places jobs across regions by integer programming, and shifts them in time within each deadline while enforcing a per-region capacity ceiling at every slot. The carbon-latency weight is chosen at each slot from the knee point of the Pareto curve. On 146,000 synthetic jobs driven by one year of measured carbon-intensity data across eight regions, CAST cuts emissions by 63.03% with no deadline violations. Ignoring capacity reaches 65.93%, but those 2.90 percentage points cost an 89-fold rise in ceiling violations, from 227 to 20,208. Enforcing capacity instead removes them but misses 3,044 deadlines; no schedule satisfies both. The key finding lies in the distribution: without a capacity limit, 71.9% of temporal-shifting savings arise in California alone, and 87.4% of its over-capacity time falls in 19:00-21:00 UTC, local midday. The region where temporal shifting creates value is the same region where capacity blocks it.

Keywords: Carbon-aware scheduling, Workload shifting, Integer programming, Regional capacity, LSTM

1. 서론

데이터센터가 쓰는 전력이 빠르게 늘고 있다. 국제에너지기구는 2024년 약 415TWh였던 전 세계 데이터센터 전력 소비가 2030년 945TWh에 이를 것으로 전망하며 AI를 가장 큰 요인으로 지목했다[1]. 에너지 사용의 투명성이 규제 대상이 된 이상[2] 배출을 줄이는 일은 운영상의 과제가 되었고, 이미 가동 중인 인프라에 신규 설비 없이 적용할 수 있는 수단이 필요하다.

배출량을 정하는 것은 소비 전력량만이 아니다. 전력 1kWh를 만들 때 나오는 탄소량, 곧 탄소집약도는 그 순간 전력망의 발전원 구성에 따라 달라진다. 본 연구의 여덟 리전에서도 2025년 실측 연평균이 인도 544.1 gCO_2/kWh, 프랑스 13.6 gCO_2/kWh로 39.9배 벌어지고, 하루 안에서도 캘리포니아는 최댓값이 최솟값의 3.53배다. 클라우드는 이미 여러 리전에 흩어져 있고 배치 작업은 완료 시점에 여유가 있으므로, 이 격차를 쓸 조건은 갖추어져 있다.

선행 연구는 두 축 중 하나에 머물러 있다. 시간 축에서는 구글의 CICS[3]가, 공간 축에서는 CASPER[4]와 VMware GSLB[5]가 대표적인데, 한 축만 다룰 뿐 아니라 탄소와 다른 목적 사이의 가중치를 고정 상수로 둔다. 두 축을 함께 다룬 Sukprasert 등의 상한 분석[6]은 공간 이동이 지배적이라고 결론지었으나 결합 분석에서는 용량 제약을 두지 않았다. 용량을 명시적으로 다룬 CarbonFlex는 단일 클러스터 내 시간 이동에 한정되고[7], 두 축과 용량을 함께 둔 Caspian[8]은 그 대신 마감을 목적함수의 페널티로 완화한다.

용량을 빼면 무슨 일이 생기는지는 본 연구의 데이터에 그대로 드러난다. Sukprasert 등의 정책을 원문 그대로 다시 구현해 돌리면 작업 146,000건 가운데 145,641건, 곧 99.75%가 프랑스 한 리전으로 몰린다(4.4절). 배출은 96.61% 줄지만 상한을 넘긴 배정이 120,186건이다. 가장 깨끗한 리전 하나로 전부 보내는 것이 최적해가 되기 때문이며, 계산 위에서만 성립하는 배치다. 용량을 제약으로 두는 순간 문제가 달라진다 — 어느 리전이 깨끗한가가 아니라, 그 리전이 받아 줄 수 있는 만큼만 보내고 남은 작업을 어디에 언제 둘 것인가가 된다.

본 논문은 탄소 인지 스케줄링 시스템 CAST를 제안한다. 예측 단계가 여덟 리전의 향후 24시간 탄소집약도를 LSTM으로 내고, 공간 이동 단계가 정수 선형 계획법으로 실행 리전을 정하며, 시간 이동 단계가 마감 이내에서 실행 시각을 정한다. 배치는 예측값으로 결정하되 배출량은 실측값으로 세어 예측 오차의 대가가 결과에 남도록 했다.

본 논문의 기여는 두 가지다. 첫째, 공간 축의 탄소–지연 가중치를 고정하지 않고 매 슬롯 파레토 무릎점에서 자동으로 정하며, 시간 축의 가중치는 작업이 신고한 지연 등급의 함수로 둔다. 평균으로는 고정값과 같은 가중치를 쓰는 셈인데도 고정 α=0.5보다 탄소를 10.64%포인트 더 줄인다. 둘째, 시간 이동이 리전 용량을 보지 않으면 동시 실행 수가 여유율 상한의 3.4배에 이른다는 것을 실측하고, 이를 온라인으로 강제하는 Algorithm 1을 제시한다. 슬롯마다 리전 점유량을 추적해 상한에 닿은 슬롯을 후보에서 빼며, 어떤 작업도 무한히 연기되지 않고 마감 위반이 없음을 증명한다. 선행 연구[7]는 이 현상을 단일 클러스터 맥락에서 지적했을 뿐 정량화하지 않았다.

두 제약이 부딪히는 자리에서 본 연구는 마감을 무를 수 없는 것으로, 용량을 양보할 수 있는 것으로 두었다. 마감을 넘긴 작업은 쓸모를 잃지만 상한을 잠시 넘긴 리전은 느려질 뿐이기 때문이다. 이 선택이 치르는 값은 4.3절에서 수치로 보인다 — 반대로 용량을 무를 수 없게 두면 상한 위반은 사라지지만 마감 위반이 3,044건 생기고 배출도 함께 는다. 둘을 동시에 지키는 배치는 이 워크로드에 존재하지 않는다.

1년치 실측 탄소집약도로 시뮬레이션한 결과 총 배출량이 63.03% 줄었고, 작업 146,000건 전량이 마감 이내에 처리되어 마감 위반은 없었다. 용량을 무시하면 65.93%까지 내려가지만 그 2.90%포인트의 대가로 상한 위반이 227건에서 20,208건으로 89배 늘어난다. 공간만 쓰면 위반이 0건, 시간만 쓰면 24건이므로 이 충돌은 두 축을 함께 쓸 때만 생긴다. 더 중요한 것은 그 부담이 고르게 퍼지지 않는다는 점이다. 용량을 두지 않았을 때 시간 이동이 만드는 절감의 71.9%가 캘리포니아 한 리전에서 나오고, 그 리전이 상한을 넘긴 866.9시간의 87.4%가 UTC 19~21시, 곧 현지 한낮에 몰린다.

2. 관련 연구

선행 연구는 탄소 인지 스케줄링이 배출을 얼마나 줄일 수 있는지를 여러 방향에서 보여 왔다. 다만 운영 조건을 제약으로 넣고 보면 세 가지가 설계에서 걸린다. 첫째, 두 축을 결합한 선행 연구[6, 8, 9]는 리전 용량과 작업 마감 가운데 하나를 양보한다. Sukprasert 등은 용량이 공간 이동의 이득을 좌우함을 스스로 확인하고도 결합 분석에서는 용량을 두지 않았고[6], Attenni 등도 자원이 항상 가용하다고 가정한다[9]. Caspian은 용량을 제약으로 두는 대신 마감을 양보하며[8], 이는 셋째에서 다룬다. 탄력적 확장으로 이를 방어할 수 있으나, 그 잉여는 클라우드 전체가 집계 수준에서 갖는 성질이지 특정 시각의 특정 리전이 갖는 성질이 아니다.

Sukprasert 등(EuroSys'24) 6.4절은 공간 이득이 낮음에도 시간 변동이 커 순 절감이 유의미한 예외로 캘리포니아와 버지니아를 지목하였다[6]. 리전 용량이 유한해 공간 이동이 조기에 포화되면 부하가 고변동 리전까지 확산되므로, 이 예외가 일반적 상황이 될 수 있다.

둘째, 탄소와 그 밖의 목적 사이의 가중치가 고정 상수다. CASPER는 탄소와 서버 프로비저닝 비용 사이의 가중치를 고정하며 저자들 스스로 다른 값에서는 결과가 달라질 것이라고 밝혔다[4]. CICS[3] 역시 탄소 비용과 피크 전력 비용의 가중합을 쓰고, 두 축을 하나의 정수 계획법으로 결합한 Caspian도 가중치를 사람이 고르며 그 값을 신중하게 정해야 한다고 결론지었다[8]. 그러나 리전 간 격차는 시각에 따라 변하므로, 옮겨서 얻는 이득이 큰 시간과 그렇지 않은 시간에 같은 가중치를 쓰는 것은 최적이라 보기 어렵다.

셋째, 두 축을 실제로 결합한 Caspian조차 작업이 무한정 밀리지 않는다는 보장을 주지 않는다. Caspian은 리전과 시간대를 함께 정하고 용량도 하드 제약으로 다루지만, 마감은 목적함수의 페널티로 두는 소프트 제약이다[8]. 본 연구는 반대로 마감을 하드 제약으로 두고 용량을 양보하며, 어떤 작업도 유한 시간 안에 처리됨을 3.4절에서 증명한다. 두 시스템은 위기 상황에서 무엇을 먼저 포기할지를 정반대로 택한 것이다.

3. CAST

3.1 개요

CAST는 예측, 공간 이동, 시간 이동의 세 단계로 구성된다. 예측 단계는 리전별로 향후 24시간의 탄소집약도를 산출한다. 공간 이동 단계는 매 1시간 슬롯마다 그 슬롯에 제출된 작업의 실행 리전을 결정한다. 시간 이동 단계는 배정된 리전 안에서 각 작업의 실행 시각을 마감 이내로 결정한다. 그림 1이 세 단계와 단계 사이를 오가는 값을 보인다.

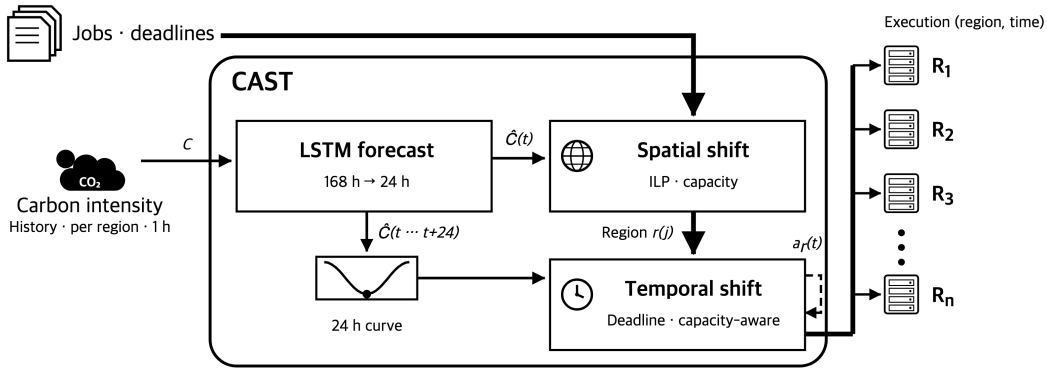


그림 1 CAST의 전체 파이프라인. 굵은 선은 작업, 가는 선은 탄소 자료, 점선은 남은 용량이 그 단계로 다시 들어가는 길이다.

Fig. 1 Overall pipeline of CAST. Thick lines carry jobs, thin lines carry carbon data, and the dashed line returns the remaining capacity to the same stage.

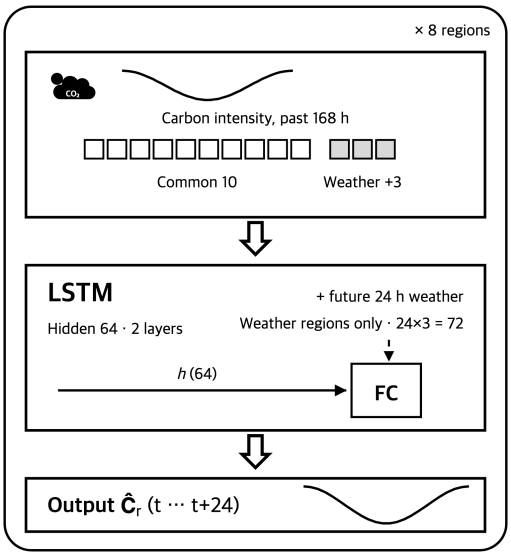


그림 2 리전별 LSTM 구조. 모델은 리전마다 독립이다.

Fig. 2 Per-region LSTM architecture. One model per region.

설계 원칙은 셋이다. 공간 이동을 먼저 적용하고 시간 이동을 보완으로 둔다 — 선행 연구는 전 세계 70% 이상의 리전이 일간 변동성이 낮아 시간 이동 단독의 전 지구 평균 절감이 8.4%에 그친다고 보고하였다[6]. 또한 CAST에서 탄소와 지연의 가중치는 고정하지 않는다. 그리고 배치 결정은 예측값으로, 배출량 산정은 실측값으로 한다. 둘 다 예측값을 쓰면 예측 오차의 대가가 결과에서 사라지기 때문이다.

예측 모델은 리전마다 독립인 2층 LSTM이며, 은닉 크기는 64다. 과거 168시간의 탄소집약도, 무탄소·재생 전력 비율, 시각, 요일, 월의 주기 변수, 공휴일 여부를 받아 향후 24시간을 한 번에 출력한다. 캘리포니아, 텍사스, 독일의 세 리전은 풍속, 일사량, 기온을 입력에 더하고, 향후 24시간의 같은 기상 변수(24×3=72개)도 출력층에 함께 넣는다. 구조는 그림 2와 같고, 그림 3은 그 모델이 실제로 낸 예측 한 구간이다. 여기까지가 예측 모듈이다. 이제부터는 그 예측을 받아 푸는 스케줄링 문제를 적는다.

3.2 표기와 형식화

작업 집합을 J, 리전 집합을 R로 둔다. 슬롯 길이는 탄소집약도 자료의 해상도에 맞춘 1시간이다. 작업 j는 출발 리전 o_j, 제출 시각 s_j, 실행 시간 d_j, 지연 등급 k_j, 지연 예산 L_j를 가지며 마감은 $D_j = s_j + L_j + d_j$다. 리전 r은 용량 cap_r을 가지고 여기에 여유율 η를 곱한 값이 동시 실행 상한이 된다. 스케줄러가 정하는 값은 배정 변수 x_{jr}과 실행 시각 τ_j 둘이다.

목적함수는 총 배출량이다 — 작업마다 자기가 실행된 리전에서 실행 시간 동안 내보낸 양을 구해 전부 더한 값이며, 이것을 최소화한다.

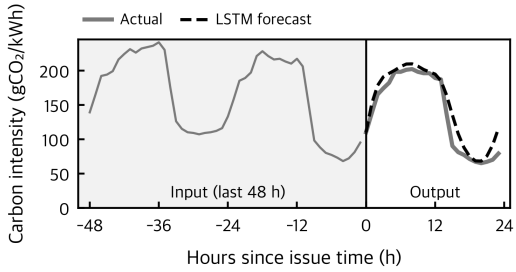


그림 3 리전별 탄소집약도 예측. 캘리포니아의 한 구간이며 평균 절대 오차는 12.2 gCO_2/kWh다.

Fig. 3 Per-region carbon intensity forecast. One window for California; the mean absolute error is 12.2 gCO_2/kWh.

$$\min \sum_{j \in J} P \int_{\tau_j}^{\tau_j + d_j} C_{r(j)}(t)\,dt \qquad (1)$$

여기서 $r(j)$는 $x_{jr} = 1$인 리전이고 P는 작업의 소비 전력이다. 제약은 네 가지다.

$$\sum_{r \in R} x_{jr} = 1, \forall j \in J \qquad (2)$$

$$\left|\{j : r(j) = r, \tau_j \le t < \tau_j + d_j\}\right| \le \lfloor \eta \cdot cap_r \rfloor, \forall r \in R, \forall t \qquad (3)$$

$$s_j \le \tau_j, \tau_j + d_j \le D_j, \forall j \in J \qquad (4)$$

$$(\tau_j - s_j) + \ell_{o_j r(j)} \le L_j, \forall j \in J \qquad (5)$$

식 (2)는 한 작업을 한 리전에만 배정한다는 뜻이다. 식 (4)는 실행이 제출 이후이고 마감 이내여야 한다는 뜻으로, 마감은 무를 수 없는 하드 제약이다 — 이를 벌점으로 무르게 둔 선행 정책에서는 마감 위반이 나온다(표 6). 식 (5)는 지연 예산을 정한다. 여기서 ℓ은 출발 리전에서 배정 리전까지의 왕복 지연이며, 전 구간에서 고정된 8×8 실측 행렬에서 읽는다(4.1절). 식 (1)~(5)를 하나의 정수 계획으로 묶지 않은 것은 시간 이동을 온라인으로 두기 위해서다 — 묶으면 모든 작업과 1년치 예측을 미리 알아야 풀린다.

식 (3)이 본 연구의 형식화가 선행 연구와 갈라지는 지점이다.

3.3 공간 이동 — 슬롯 단위 정수 계획 라우팅

공간 이동 단계는 매 슬롯마다 그 슬롯에 제출된 작업을 모으고, 리전별 탄소집약도 예측을 받아 탄소 비용과 지연 비용을 정규화한 뒤, 정수 계획을 풀어 배정을 확정하고 실행 시간 동안 해당 리전의 용량에 반영한다.

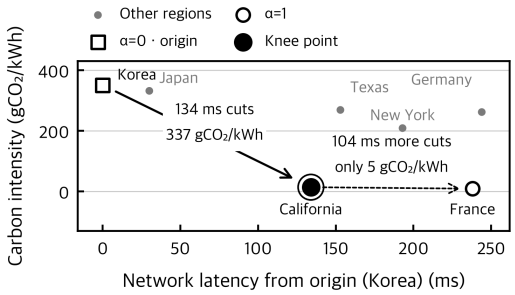


그림 4 공간 이동이 맞바꾸는 두 축. 한 슬롯의 여덟 리전이다(한국 출발).

Fig. 4 Trade-off axes of spatial shifting. Eight regions at one slot, with Korea as the origin.

두 비용은 서로 다른 방식으로 정규화한다. 탄소 비용은 슬롯마다 그 슬롯의 리전별 최댓값으로 나누고, 지연 비용은 전 구간에서 고정된 값인 지연 행렬의 최댓값 244ms로 나눈다.

$$\tilde{C}_r = \frac{\hat{C}_r}{\max_{r'} \hat{C}_{r'}}, \tilde{\ell}_{or} = \frac{\ell_{or}}{244} \qquad (6)$$

탄소 비용만 슬롯별로 정규화하는 것은 슬롯 간 격차의 절대 크기를 가중치 해석에서 떼어내기 위함이다. 고정된 분모를 쓰면 전 리전이 함께 깨끗한 슬롯에서 탄소 항이 일률적으로 작아져 같은 가중치가 슬롯마다 다른 뜻을 갖는다.

슬롯의 배정 문제는 다음과 같다. 배치에 속한 작업을 j, 리전을 r, 배정 변수를 x_{jr}, 미배정 변수를 z_j로 둔다.

$$\min \sum_j \sum_r \left(\alpha \tilde{C}_r + (1-\alpha)\tilde{\ell}_{o_j r}\right) x_{jr} + P_{slack} \sum_j z_j \qquad (7)$$

$$s.t. \quad \sum_r x_{jr} + z_j = 1 \forall j, \sum_j x_{jr} \le a_r \forall r \qquad (8)$$

a_r은 여유율을 적용한 용량에서 실행 중인 작업 수를 뺀 값이다. 미배정 변수 z_j는 어느 리전에도 배정할 수 없을 때 문제가 실행 불가능해지는 것을 막으며, P_{slack}에 어떤 배정 비용보다 큰 값을 주어 웬만해선 선택되지 않게 한다. 실측에서 미배정은 한 건도 없었다.

가중치 α는 슬롯마다 파레토 무릎점으로 결정한다. 0부터 1까지 0.1 간격의 후보 11개에 대해 각각 배정 문제를 풀어 그 배정의 평균 네트워크 지연과 예상 배출량을 얻고, 두 축을 각각 0에서 1로 정규화한 뒤 원점에 가장 가까운 점의 α와 배정을 택한다.

$$\alpha^* = \arg\min_\alpha \sqrt{\tilde{\lambda}(\alpha)^2 + \tilde{\mu}(\alpha)^2} \qquad (9)$$

여기서 $\tilde{\lambda}$와 $\tilde{\mu}$는 각각 0–1로 정규화한 평균 지연과 예상 배출이다.

미배정이 발생하는 후보가 있으면 미배정 수가 최소인 후보들 안에서만 고르고, 거리가 같으면 작은 α를 택한다.

평균은 같은 가중치(α≈0.508)를 쓰는 셈이지만 매 슬롯 다시 계산한다는 점이 차이를 만든다. 고정 α=0.5는 15,718.6 kg(−46.22%, 평균 지연 29.5 ms)을, 무릎점 자동 선택은 12,609.8 kg(−56.85%, 36.2 ms)을 낸다. 지연을 6.7 ms 더 쓰고 탄소를 10.64%포인트 더 줄인 것이다.

가중치가 배정을 어떻게 바꾸는지는 한 슬롯을 열어 보면 분명하다. 그림 4의 슬롯에서 α를 0에서 1까지 옮기면 목적지가 홈(출발 리전, 여기서는 한국), 캘리포니아, 프랑스의 세 구간으로 갈린다(표 1). 무릎점 0.508은 가운데 구간에 들어 탄소를 홈 대비 96% 줄이면서 지연은 상한의 55%만 쓴다. 1년 전체의 흐름은 그림 5에 있다 — 프랑스와 캘리포니아가 전체의 62.3%를 받고, 독일은 자기 작업의 73.6%를, 인도는 71.0%를 프랑스로 보낸다.

표 1 가중치 α 에 따른 배정 변화. 그림 4와 같은 슬롯이며 탄소는 gCO_2/kWh, 지연은 ms다.

Table 1 Assignment change by weight α. Same slot as Fig. 4; carbon in gCO_2/kWh, latency in ms.

α	Assigned region	Carbon	Latency
0.0–0.4	Korea (origin)	351	0
0.5–0.9	California	14	134
1.0	France	9	238

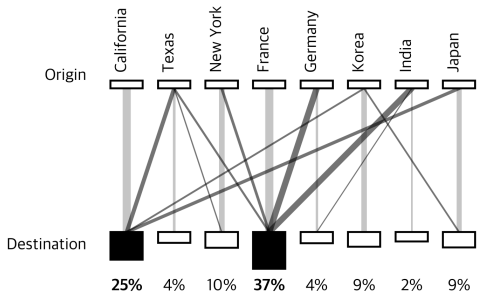


그림 5 공간 이동이 만든 리전 간 흐름. 1년치 146,000건 가운데 옮긴 물량의 91%를 차지하는 흐름 10개다. 회색 세로선은 홈에 남은 몫, 검은 칠은 20% 넘게 받은 리전이다.

Fig. 5 Inter-region flow created by spatial shifting. The 10 flows carrying 91% of moved jobs out of 146,000 assignments over one year. Gray verticals are the share kept at the origin, black fill marks regions receiving over 20%.

고정 상수는 실패 방향이 슬롯에 따라 반대다. 격차가 큰 슬롯에서는 옮길 이득을 남긴 채 보수적으로, 작은 슬롯에서는 얻을 탄소가 적은데도 지연을 지불하며 공격적으로 움직인다. 무릎점은 정규화한 곡선의 기하만 쓰므로 사람이 정한 가중치를 요구하지 않는다. 그림 6은 고정 α 곡선과 슬롯별 무릎점을 함께 보인다. 무릎점은 슬롯마다 α가 달라 이 곡선 위의 점이 아니며, 어떤 고정값으로도 대신할 수 없다. 또 탄소가 가장 낮은 지점도 아니다 — α=0.75는 10,396.5 kg으로 더 낮지만 지연이 62.6 ms로 무릎점의 1.7배다. 고정값 전 구간은 4.2절 표 4에 있다.

3.4 시간 이동 — 용량 인지 슬롯 선택

시간 이동 단계는 작업 j의 실행 가능 윈도우를 $[t_{early}, t_{late}] = [s_j, D_j - d_j]$로 두고 그 안의 1시간 슬롯을 후보로 삼는다. 정수 슬롯이 제출 시각 하나뿐이면 옮길 자리가 없으므로 즉시 실행한다(Algorithm 1의 4번 줄). 이 경로는 용량을 보지 않는다 — 미룰 수 없는 작업은 막을 방법이 없다. 후보는 예측 지평 안으로 제한한다. 예측이 24시간까지만 있어 그 너머는 비교할 근거가 없기 때문이다.

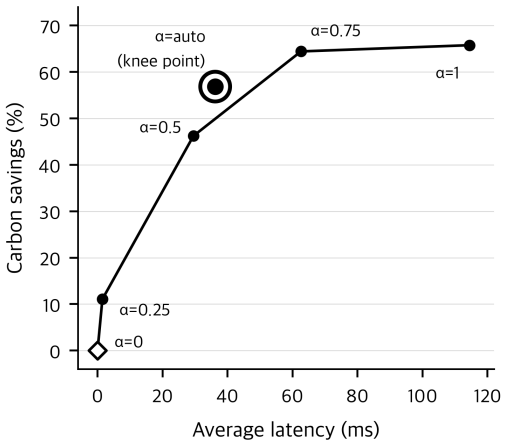


그림 6 고정 α 스윕과 슬롯별 무릎점(α=auto)의 파레토 곡선. 선은 고정 α 점만 잇는다.

Fig. 6 Pareto curve of the fixed-α sweep and per-slot knee points. The line connects only the fixed-α points.

후보 슬롯 t의 점수는 정규화한 탄소 비용과 지연 비용의 가중합이다.

$$\bar{C}(t) = \frac{\hat{C}(t) - C_{\min}}{C_{\max} - C_{\min}}, \bar{D}(t) = \frac{t - t_{early}}{t_{late} - t_{early}} \quad (10)$$

$$score(t) = \beta(k)\bar{C}(t) + (1 - \beta(k))\bar{D}(t), \beta(k) = \frac{6-k}{5} \quad (11)$$

$\hat{C}(t)$는 t부터 실행 시간 동안의 예측 탄소집약도 평균이고, $Q(t)$는 그것을 후보 집합 안에서 0–1로 옮긴 값이며 C_{min}·C_{max}는 그 집합의 최솟값과 최댓값이다. 가중치는 작업이 신고한 지연 등급의 함수로 둔다. 등급 k가 작을수록, 곧 여유가 클수록 탄소 항의 비중이 커져 가장 깨끗한 슬롯이 뽑히고, 등급이 높으면 지연 항이 지배해 사실상 즉시 실행된다.

여기까지는 각 작업이 제 최적 슬롯을 따로 고르는 규칙이라, 식 (3)의 리전별 동시 실행 상한은 아직 강제되지 않았다. 그림 7이 그 상태의 하루를 보인다 — 옮기고 싶은 저탄소 시각이 이미 상한에 닿아 있다. 식 (3)의 우변 $\lfloor \eta \cdot cap_r \rfloor$을 K_r로, 예측 지평을 H로 줄여 쓴다. K_r을 유효 상한이라 부른다.

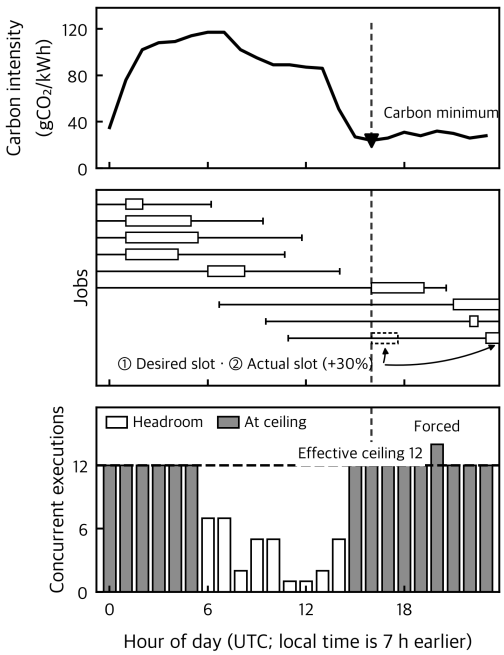


그림 7 용량이 시간 이동을 막는 방식. 캘리포니아의 한 날(4월)이며 창이 가장 넓은 9건이다. 가는 선이 창, 굵은 막대가 실행 구간이고 세로선은 탄소 최저이자 상한 도달 시각이다.

Fig. 7 How capacity blocks temporal shifting. One April day for California, showing the 9 jobs with the widest windows. Thin lines are windows, thick bars the executed interval, and the vertical marks the carbon minimum, which is also when the ceiling is reached.

먼저 리전 r, 시각 t에서 남은 자리를 정의한다. 3.3절의 a_t에 시각을 붙인 것이다.

$$n_r(t) = \left|\{j : r(j) = r,\ \tau_j \le t < \tau_j + d_j\}\right|,$$
$$a_r(t) = K_r - n_r(t) \qquad (12)$$

식 (3)을 남은 자리 형태로 다시 쓴 것이다. 작업을 t'에 놓으려면 시작 시점만이 아니라 실행이 끝날 때까지 매 순간 자리가 있어야 한다.

다음으로, 슬롯 t에서 지금이 남은 탐색 구간 안에서 최선인 작업들을 고른다.

$$Q(t) = \{j : s_j \le t,\ \tau_j \text{ 미정}\}$$

$$P_r(t) = \left\{ j \in Q(t) : r(j) = r,\ \arg\min_{t'} score_j(t') = \max(t, s_j) \right\} \qquad (13)$$

점수는 식 (11)을 그대로 쓰되 예측은 매 슬롯 새로 발행된 최신값을, 지금 슬롯만은 관측값을 쓴다. t'는 남은 윈도우 $W_j(t) = [\max(t, s_j), \min(D_j - d_j, t+H)]$ 안의 슬롯이다. 제출 시점의 예측을 끝까지 쓰는 사전 예약형과 달리 미래를 안다고 가정하지 않는다.

자리가 $P_r(t)$의 크기보다 적으면 누구를 먼저 넣을지 정해야 한다. 남은 여유가 가장 적은 작업을 먼저 넣는다 — 순수 EDF(Earliest Deadline First)다.

$$u_j(t) = (D_j - d_j) - t$$
$$F_r(t) = \{j \in Q(t) : r(j) = r,\ u_j(t) < 1\}$$
$R_r(t) = P_r(t) \setminus F_r(t)$를 $u_j(t)$ 오름차순 정렬
$$S_r(t) = F_r(t) \cup R_r(t)[1 \dots \max(a_r(t) - |F_r(t)|,\ 0)] \qquad (14)$$

$j \in S_r(t)$이면 $\tau_j \leftarrow \max(t, s_j)$로 확정한다.

세 가지를 밝혀둔다. 우선순위는 탄소 이득이 아니라 남은 여유다 — 탄소 이득으로 정렬하면 나중에 성과로 보고할 바로 그 값으로 지금 순서를 정하는 셈이라 결과가 좋아 보이는 것이 당연해진다. $F_r(t)$는 대기 집합 전체에서 고른다. $|F_r(t)|$가 $a_r(t)$보다 많으면 그 슬롯은 의도적으로 상한을 넘긴다 — 마감은 하드 제약, 용량은 소프트 제약이며 둘이 충돌하면 마감을 먼저 지킨다. 반대로 골랐을 때 무엇을 잃는지는 4.3절에서 대조군으로 잰다.

Algorithm 1은 이 규칙을 슬롯 단위로 반복한다.

```
Algorithm 1  Slot-wise rolling re-evaluation
Input: K_r, Ĉ_r^(t)(·), H, job set
Output: τ_j for all j
 1: Q ← ∅
 2: for t ← 0 to t_max do
 3:   for each j with ⌊s_j⌋ = t do
 4:     if ⌊D_j − d_j⌋ ≤ ⌊s_j⌋ then τ_j ← s_j
 5:     else Q ← Q ∪ { j }
```

```
 6:   for each r ∈ R do
 7:     a_r(t) ← K_r − n_r(t)
 8:     F ← { j ∈ Q : r(j)=r, u_j(t) <
1 }
 9:     P ← ∅
10:     for each j ∈ Q with r(j)=r, j ∉
F do
11:       W_j(t) ← feasible window
12:       if argmin_(t'∈W_j(t))
score_j(t')
             = max(t, s_j) then
13:         P ← P ∪ {j}
14:     sort P by u_j(t) ascending
15:     S ← F ∪ P[1 … max(a_r(t) − |F|,
0)]
16:     for each j ∈ S do
17:       τ_j ← max(t, s_j); Q ← Q \ {j}
18:   if Q = ∅ and no arrivals remain
then break
```

이 알고리즘은 두 가지를 보장한다. 슬롯이 하나 지날 때마다 $u_r(t)$는 정확히 1씩 줄고 시작 값이 유한하므로 유한한 슬롯 안에 $F_r(t)$에 편입되어 용량과 무관하게 실행이 확정된다 — 어떤 작업도 무한히 연기되지 않는다. 또 후보 슬롯이 애초에 $D_j - d_j$ 이하로만 잡히므로 마감을 넘긴 배정은 탐색 공간에 없다.

슬롯당 계산 비용은 그 슬롯의 대기 작업 수와 예측 지평 H의 곱에 비례한다.

비교를 위해 같은 점수 규칙을 쓰되 용량을 전혀 보지 않는 경우도 함께 잰다. 후보 슬롯을 고를 때 이미 배정된 작업 수를 보지 않으면 저탄소 시간대로 몰려도 아무것도 막지 않는다. 이 값이 4장의 반사실 상한이고, 실제로 달성한 값과의 차이가 곧 용량 제약의 대가다. 4장에서는 이를 용량 무시라 부른다.

3.5 탄소 회계

작업 j의 배출량은 실행 리전의 실측 탄소집약도를 실행 구간에 대해 적분한 값에 소비 전력을 곱한 값이다.

$$E_j = P \int_{\tau_j}^{\tau_j + d_j} C_{r(j)}(t)\,dt \qquad (15)$$

실측 계열이 1시간 해상도이므로 적분은 슬롯 단위로 근사한다 — 실행이 걸치는 슬롯들의 평균 강도에 실행 시간을 곱한다. 겹침 길이로 가중해 정확히 적분해도 총 배출량은 0.001% 안에서 같다. 소비 전력은 작업당 1kW 상수로 둔다. 상대 비교를 위한 단순화이며, 그 덕에 배출량 차이는 전적으로 실행 리전과 시각에서 온다.

4. 실험과 분석

4.1 실험 설정

평가는 8개 리전의 2025년 실측 탄소집약도 1년치 위에서 수행한다. 탄소집약도는 Electricity Maps API를 통해 1시간 해상도로 수집하였다[10] — 발전과 설비를 모두 포함하는 전과정 배출계수 체계를 쓰는 공식 방법론이다[11].

워크로드는 총 146,000건이며 리전당 하루 50건이 제출된다. 도착률과 등급별 분포(k=1부터 3:1:1:2:3)는 실측 트레이스를 구하지 못해 설정한 합성 파라미터다. 표 2에서 보듯 k=3 이상이 전체의 60.1%인 87,712건으로, 지연 예산이 슬롯 길이 1시간보다 짧아 시간 이동의 여지가 거의 없다. 옮기기 쉬운 작업만 골라 담은 워크로드가 아니다.

무엇이 실측이고 무엇이 가정인지 갈라 둔다. 여덟 리전의 1년치 탄소집약도와 8×8 왕복 지연 행렬은 실측값이고, 작업의 도착률과 지연 등급 분포, 실행 시간 분포, 리전별 용량은 설정값이다. 탄소 인지 스케줄링을 위해 공개된 워크로드 트레이스가 없어 합성할 수밖에 없었다. 다만 평가 구간은 이 분야에서 짧은 편이 아니다 — 비교 대상인 CASPER와 Caspian은 각각 24시간을 평가 구간으로 삼았고 Caspian의 작업 수는 200건이다. 계절에 따라 갈리는 결과는 하루치 자료로는 볼 수 없다.

표 2 지연 등급과 가중치. 예산은 등급 안에서 로그균등으로 뽑는다.

Table 2 Latency tiers and weights. Budgets are drawn log-uniformly within each tier.

Tier k	Latency budget	Jobs	β(k)
1	12–24 h	43,830	1.0
2	2–8 h	14,458	0.8
3	2–10 min	14,716	0.6
4	15–60 s	29,244	0.4
5	1–5 s	43,752	0.2

슬롯 길이는 1시간이지만 온라인 시간 이동은 슬롯 경계가 아니라 제출 시각을 기준으로 굴러가므로, 지연 예산이 1시간보다 짧은 k=3, 4, 5도 대상에서 빠지지 않는다. 실제로 이 세 등급에서 537건이 지연되었고 마감을 넘긴 작업은 없었다.

작업 실행 시간은 지연 등급마다 로그정규분포에서 뽑아 등급별 범위로 자른다. 중앙값은 k=1부터 5까지 90분, 30분, 3분, 30초, 5초이고 로그표준편차는 0.9, 0.9, 0.8, 0.7, 0.6이며, 자르는 범위는 차례로 5분~6시간, 1분~4시간, 5초~30분, 1초~5분, 1초~1분이다. 지연 예산은 표 2의 등급별 구간 안에서 로그균등으로 뽑는다.

제출 시각은 현지 시각으로 정하는데, k=3, 4, 5는 낮 13시를, k=1, 2는 새벽 2시를 봉우리로 하는 표준편차 3시간의 정규분포에서 70%를 뽑고 나머지 30%는 반대 시간대에서 균등하게 뽑는다. 난수 시드는 42이며, 이 규칙을 담은 생성기와 그것이 만든 작업 명세를 저장소에 함께 둔다.

네트워크 지연은 8×8 리전 간 왕복 지연 실측 행렬(최댓값 244 ms, 비대각 최솟값 12 ms)을 Azure 통계[12]에서 가져왔다. 리전 용량은 여덟 리전에 균일하게 준다 — 기준 배정의 리전별 최대 동시 실행 수에 1.2를 곱해 16을 얻고, 여유율 0.8을 적용해 유효 상한 12를 쓴다. 기준 배정이 도달한 최대치(13건)의 92%이며, 이 값을 바꾼 결과는 4.3절에 있다. 하루 앞 예측은 선행 연구가 따로 다룬 문제이고[13] 본 연구의 기여가 아니므로 예측 벤치마크는 하지 않았다. 예측 오차는 지평 1~24시간을 모두 평균하여 여덟 리전 평균 26.1 gCO_2/kWh이고 24시간 지평만 보면 28.5 gCO_2/kWh이며, 예측이 어려운 리전이 곧 시간 이동의 이득이 큰 리전이다.

예측 모델의 학습 조건은 이렇다. 2021~2023년으로 학습하고 2024년으로 검증했으며 2025년으로 시험하였다 — 연도 단위로 갈라 시간 순서를 지켰으므로, 이 실험이 쓰는 1년치는 모델이 학습에서 보지 못한 구간이다. 탄소집약도와 기상 변수는 리전마다 학습 구간만으로 적합한 최소최대 정규화를 거친다. 손실 함수는 정규화 값의 평균제곱오차이고, 최적화기는 Adam(학습률 10^{-3}, 가중치 감쇠 10^{-4})이며 기울기는 노름 1.0에서 자른다. 최대 150 에폭을 돌리되 검증 평균절대오차가 15 에폭 동안 나아지지 않거나 최적 에폭에서 30 에폭이 지나면 멈추고, 검증 손실이 8 에폭 정체하면 학습률을 절반으로 줄인다. 최적 에폭은 리전마다 53에서 148 사이였다. 난수 시드는 42다.

4.2 주요 결과

본 연구의 다섯 방식을 같은 회계로 비교하였다. 기준은 홈 리전 즉시 실행, 시간만은 홈을 고정하고 실행 시각만 옮긴 것, 공간만은 리전만 옮긴 것이다. CAST는 공간만에 시간 이동을 더하고 매 슬롯 리전 용량을 확인하며, 용량 무시는 같은 조합에서 용량을 보지 않은 경우다 — 배출은 더 낮지만 상한을 20,208번 넘기므로 실제로 돌릴 수 있는 배치가 아니다. 이 다섯 이름을 이후 모든 표와 그림에서 그대로 쓴다. 결과는 표 3에 있고, 그림 8이 같은 값을 막대로 견준다.

표 3 다섯 방식의 총 배출량과 용량 위반. 위반은 유효 상한 12를 넘겨 배정한 건수이며, 상한이 기준 배정의 최대치에서 나온 값이라 기준에서도 1건이 잡힌다.

Table 3 Total emissions and ceiling violations across the five methods. A violation is an assignment past the effective ceiling of 12; the ceiling derives from the baseline's own peak, so the baseline itself shows one.

Method	Total emissions (kg)	vs. baseline	Ceiling violations
Baseline	29,225.6	—	1
Temporal-only	23,789.1	−18.60%	24
Spatial-only	12,609.8	−56.85%	0
CAST	**10,805.0**	**−63.03%**	**227**
Capacity ignored	9,958.2	−65.93%	20,208

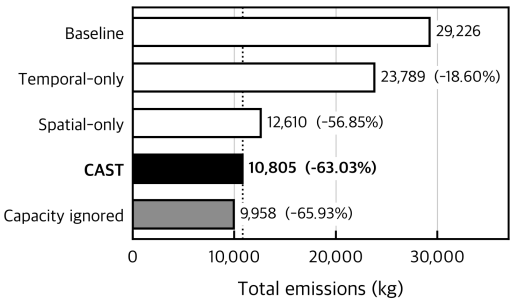


그림 8 다섯 방식의 총 배출량 비교 (표 3).

Fig. 8 Total emissions across the five methods (Table 3).

공간 이동만으로 56.85%가 줄고, 시간 이동이 용량을 보지 않으면 65.93%까지 내려가지만 이는 실현 불가능한 반사실이다. 배치 뒤 사후로 용량을 강제하면 59.37%에 그친다(절차는 4.3절). CAST는 두 경계 사이의 63.03%를 매 슬롯 용량을 확인하며 얻으며, 그 과정은 4.3절에서 다룬다. 작업당 1kW 가정에서 1년에 18,420.6 kg을 덜 내보낸다는 뜻이다.

위반이 어디서 오는지 보면 조합 자체가 문제를 만든다는 것이 드러난다. 공간만 쓰면 상한 위반이 0건이고 시간만 쓰면 24건인데, 둘을 함께 쓰면 20,208건이 된다. 공간 이동은 작업을 깨끗한 리전으로 모으고 시간 이동은 그 리전 안에서 다시 깨끗한 시각으로 모으는데, 두 쏠림이 같은 자리에서 겹치기 때문이다. 본 연구가 푸는 것은 두 축을 결합하는 일 자체가 아니라 결합이 만들어 낸 이 충돌이다.

작업 146,000건은 전량 마감 이내에 처리되었다. 시간 이동이 후보 슬롯을 마감 이내에서만 찾으므로 설계상의 결과다. 공간 이동에 따른 평균 네트워크 지연은 36.2 ms, 95 백분위수는 137 ms이고 홈 리전 처리 비율은 56.3%다.

가중치를 고정했다면 어땠을지도 함께 재었다. α를 0에서 1까지 쓸어 각 값의 배정을 같은 회계로 채점한 것이 표 4다. α를 올리면 탄소가 줄고 지연이 느는 것은 설계상 당연하므로, 이 표가 답하는 것은 어느 고정값이 가장 적게 내보내는가가 아니라 무릎점이 어느 자리에 서는가다. 무릎점은 평균 α 0.508로 고정 0.5와 사실상 같은 가중치를 쓰면서 배출을 10.64%포인트 더 줄이고 지연은 6.7 ms만 더 쓴다. 더 낮은 배출을 내는 고정값도 있으나 치르는 지연이 다르다 — α=0.75는 10,396.5 kg이지만 지연이 62.6 ms로 무릎점의 1.7배이고, α=1.0은 10,004.4 kg에 114.4 ms로 3.2배다.

표 4 고정 가중치 α 전 구간과 슬롯별 무릎점. 같은 회계로 채점했으며 지연은 공간 이동이 더한 평균 왕복 지연이다. 홈 리전 처리 비율은 α=0부터 차례로 100.0, 88.2, 59.0, 39.6, 13.0%이고 무릎점은 56.3%다.

Table 4 Full sweep of the fixed weight α against the per-slot knee point, scored under the same accounting. Latency is the mean round trip added by spatial shifting.

α	Emissions (kg)	vs. baseline	Avg latency (ms)
0	29,225.4	0.00%	0.01
0.25	25,989.6	−11.07%	1.52
0.5	15,718.6	−46.22%	29.53
0.75	10,396.5	−64.43%	62.61
1.0	10,004.4	−65.77%	114.40
auto (knee)	12,609.8	−56.85%	36.22

무릎점이 고른 α의 분포를 보면 고정값이 왜 대신할 수 없는지 드러난다. 0.4·0.5·0.6 세 값이 전체 슬롯의 73.7%를 차지하지만 0.0에서 0.9까지 흩어지고, α=1.0은 한 번도 뽑히지 않았다. 시각대로 보면 UTC 2시의 0.432에서 14시의 0.577까지 오르내린다 — 태양광이 들어와 리전 간 격차가 벌어지는 한낮에는 탄소 쪽으로 기울고, 전 리전이 함께 어두운 새벽에는 지연 쪽으로 돌아온다. 고정 상수는 이 왕복을 따라갈 수 없다. α는 슬롯마다 하나를 골라 그 슬롯의 여덟 리전에 함께 적용하므로 리전별로 다른 α는 존재하지 않는다. 리전에 따라 갈리는 것은 가중치가 아니라 받는 물량이며, 그 분포는 그림 5와 표 4의 홈 리전 비율에 있다.

시간 이동의 절감은 지연 등급에 따라 극단적으로 치우친다.

절감의 93.79%가 k=1에서, 6.20%가 k=2에서 나온다. k=3, 4, 5는 지연 예산이 슬롯 길이 1시간보다 짧아 옮길 자리가 거의 없어 셋을 합쳐도 0.1 kg에 그치는데, 이들이 전체의 60.1%인 87,712건이다. 슬롯 길이를 줄이면 이 셋의 기여도 늘겠으나 정수 계획을 푸는 횟수가 그만큼 늘어난다.

4.3 용량 제약과 시간 이동

시간 이동 단계가 리전 용량을 고려하지 않을 때 무슨 일이 생기는지 측정하였다. 동시 실행 수는 모든 작업의 시작·종료 시각에서의 누적합으로 세며, 로드밸런서가 용량을 산정할 때 쓰는 정의와 같다. 용량을 보지 않으면 동시 실행이 최대 41건까지 올라 유효 상한 12의 3.4배가 되는데, 그 초과가 여덟 리전에 고루 퍼지지 않고 캘리포니아와 프랑스 두 곳에 몰린다.

두 리전의 성격은 상반된다. 캘리포니아는 상한을 넘긴 866.9시간의 87.4%가 UTC 19~21시, 곧 현지 한낮 세 시간에 몰린다(균등하다면 12.5%). 반면 프랑스는 초과가 스물네 시간대에 흩어져 가장 큰 시간대도 12.1%다. 캘리포니아의 초과는 태양광이 만드는 좁은 저탄소 시간대로의 집중이고, 프랑스의 초과는 리전이 항상 낮아 물량이 몰리는 것이다.

이 초과가 얼마나 손해인지 가늠하기 위해 배치 시점에 용량을 전혀 모르는 대조군을 두었다. 상한을 넘긴 작업을 초과가 사라질 때까지 되돌리는 규칙으로, 용량을 아는 스케줄러보다 항상 같거나 나쁘므로 손해의 하한이 된다 — 이하 순진한 하한이라 부른다. 21.4%인 31,250건이 되돌려지고 절감률은 65.93%에서 59.37%로 내려가며, 손실의 94.3%가 캘리포니아에서 나온다. 이득도 같은 리전에 쏠려, 용량을 두지 않았을 때의 절감 2,651.6 kg 가운데 71.9%가 캘리포니아 몫이다. 되돌린 작업 한 건당 손실도 캘리포니아 109.74 *g*, 프랑스 4.09 *g*으로 27배 차이가 난다.

이 손해를 줄이기 위해 용량 인지 온라인 시간 이동을 구현했다. 슬롯별로 리전 점유량을 추적해 상한에 닿은 슬롯을 후보에서 미리 뺀다. 총 배출량 10,805.0 kg, 기준 대비 63.03%로 순진한 하한보다 3.66%포인트 더 줄이면서 최대 동시 실행이 41건(유효 상한의 3.4배)에서 17건으로, 캘리포니아의 초과 시간이 866.9시간에서 7.25시간으로 줄어든다(그림 9). 상한 위반은 반사실의 20,208건에서 227건으로 89배 줄고, 남은 227건은 마감 직전이라 강제 편입된 106건과 옮길 자리가 없어 즉시 실행된 121건이다. 정상 경로의 위반은 0건이다.

유효 상한을 바꾸면 이 결과가 상한값에 얼마나 매였는지 드러난다(표 5). 6건으로 조이면 절감률이 58.05%로 내려가고, 미룰 수 없는 작업 때문에 그 상한 자체를 29,962번 넘긴다. 반대로 풀면 65.80%까지 오르지만 유효 상한 12건으로 다시 세면 위반이 25,992건이다. 12건은 그 사이에서 63.03%, 227건에 해당한다. 표 5의 무제한 행은 온라인 알고리즘에 상한만

걸지 않은 것이어서, 작업 전체를 미리 보고 푸는 표 3의 용량 무시와는 다른 값이다(9,996.2 대 9,958.2 kg).

표 5 유효 상한을 바꾸어 전 과정을 다시 돌린 결과. 위반은 여섯 행을 모두 상한 12로 다시 세어 견준 값이다. 각 행의 상한 자체로 세면 차례로 29,962 · 3,565 · 227 · 17 · 4 · 0건이며, 마감 위반은 여섯 행 모두 0건이다.

Table 5 Full pipeline rerun across effective ceilings. Violations are re-scored at the common ceiling of 12 so that the rows are comparable; counted at each row's own ceiling they are 29,962, 3,565, 227, 17, 4 and 0. Deadline violations are zero in all six rows.

Ceiling	Emissions (kg)	vs. baseline	Violations at 12
6	12,260.3	−58.05%	484
9	11,552.8	−60.47%	328
12 (CAST)	10,805.0	−63.03%	227
18	10,127.1	−65.35%	22,037
24	10,018.0	−65.72%	27,345
Unbounded	9,996.2	−65.80%	25,992

상한이 아니라 부하를 바꾸어도 같은 질문을 던졌다. 작업 수를 절반과 두 배로 바꾸어 전 과정을 다시 돌리면 절감률이 65.52%, 63.03%, 58.05%로 내려가고 상한 위반은 14건, 227건, 59,924건으로 는다. 마감 위반은 세 경우 모두 0건이다. 상한을 12로 묶어 둔 채 작업만 늘렸으니 방향은 당연하며, 두 배 규모의 59,924건은 알고리즘이 무너진 것이 아니라 용량이 모자란 상태를 잰 값이다. 용량이 부하를 따라 늘면 결과도 따라 돌아온다 — 표 5에서 상한을 24로 올리면 위반이 4건이다.

반대로 용량을 무를 수 없게 두면 어떻게 되는지도 재어 보았다. 마감이 임박한 작업의 편입과 미룰 수 없는 작업의 즉시 실행을 모두 빈자리까지만 허용하고, 자리가 없으면 다음 슬롯으로 미루게 한 판이다. 상한 초과 배정은 0건이 되고 상한을 넘긴 시간도 0이 된다. 그 대신 마감 위반이 3,044건 생기고 배출도 10,891.9 kg, 기준 대비 62.73%로 올라 CAST보다 나빠진다. 용량과 마감을 동시에 지키는 배치는 이 워크로드에 없다. 둘 중 무엇을 양보할지만 고를 수 있으며, 본 연구가 마감을 고른 것은 마감을 넘긴 작업은 쓸모를 잃지만 상한을 잠시 넘긴 리전은 느려질 뿐이기 때문이다.

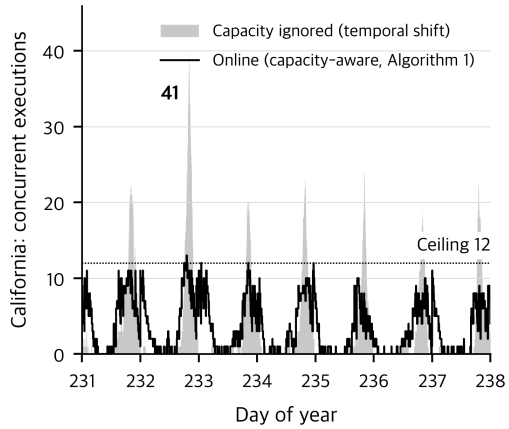


그림 9 시간 이동 전후 캘리포니아의 동시 실행 수. 연중 한 주(day 231–238)이며, 온라인의 연중 최대 17건은 다른 주에서 난다.

Fig. 9 California concurrent executions before and after temporal shifting. One week of the year (day 231–238); the online peak of 17 over the full year falls in a different week.

4.4 선행 정책 재현 비교

CICS, CASPER, Sukprasert, Caspian의 정책을 원문의 수식과 공개 코드 그대로 본 연구의 데이터 위에 다시 구현하고 같은 회계(식 (15))로 채점하였다. 결과는 표 6에, 같은 값을 두 축에 놓은 것은 그림 10에 있다. 본 연구의 네 행은 같은 종류의 선행 정책과 짝지어 읽는 자리다. 정책마다 무엇을 원문에서 그대로 가져왔고 무엇을 본 연구가 정해야 했는지는 부록 A에 적었다 — 재구현이 어느 쪽에 유리하게 기울지 않았음을 보이는 자리다.

절감률만 보면 CASPER(95.28%)와 Sukprasert 공간+시간(96.61%)이 본 연구(63.03%)를 앞선다. 뒤엣것은 1년치 탄소집약도를 모두 미리 안다고 가정한 정책이다. 다만 본 연구를 앞서는 네 방식(용량 무시, Sukprasert 시간, CASPER, Sukprasert 공간+시간)은 모두 상한 위반이 20,208건 이상이고, 그 안에서는 절감률이 높을수록 위반도 많다. 선행 정책 가운데 가장 적은 것도 49,356건으로 본 연구의 217배다. 상한 12는 기준 배정이 도달한 최대치의 92%이므로, 넘기는 배정은 그 워크로드가 한 번도 요구한 적 없는 동시 실행 수를 전제한다.

같은 축으로 좁혀 보면 결과가 뒤집힌다. 홈을 고정하고 시간만 옮기는 조건에서 CICS는 13.76%에 그쳐 본 연구의 18.60%에 못 미친다. 용량 위반은 반대로 CICS가 0, 본 연구가 24건(전체의 0.016%)이다. CICS는 하루 전에 산정한 자원 곡선 하나로 다음 날 전체를

처리해 개별 작업의 마감을 보지 못하는 반면, 본 연구는 슬롯마다 남은 여유를 다시 계산한다. Caspian은 30.51%를 내지만 17,054건이 마감을 넘긴다 — 2절에서 지적한 소프트 마감의 대가다. 다만 원문의 평가 규모는 24시간 200건이다.

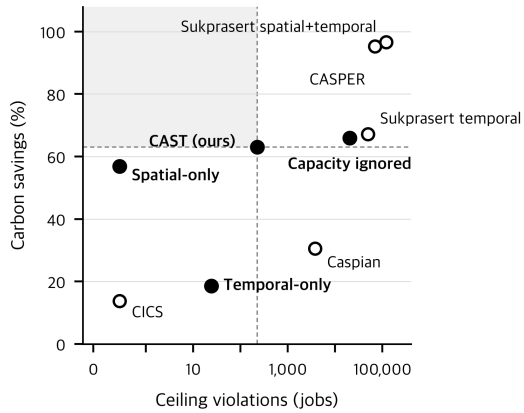


그림 10 선행 정책 재현 결과 비교 (표 6). ● 본 연구, ○ 선행 정책. 기준은 빼고 그렸으며, 음영은 CAST보다 위반이 적으면서 더 줄인 영역이다.

Fig. 10 Comparison of reproduced prior policies (Table 6). ● this work, ○ prior policy. The baseline is omitted; the shaded area would hold any policy with fewer violations and larger savings than CAST.

표 6 선행 정책을 같은 회계로 재현한 결과. 총배출은 kg, 위반은 건이다. 용량 위반은 유효 상한 12를 모든 정책에 같게 적용해 셌다.

Table 6 Prior policies reproduced under the same accounting. Emissions in kg, violations in jobs. The same effective ceiling of 12 is applied to every policy.

Policy	Total emissions	Carbon savings	Ceiling violations
Baseline	29,225.6	—	1
CICS Virtual Capacity Curve	25,203.4	13.76%	0
Caspian (fractional)	20,308.4	30.51%	3,745
Sukprasert temporal	9,583.6	67.21%	49,356
CASPER CAP	1,379.1	95.28%	69,310
Sukprasert spatial+temporal	990.8	96.61%	120,186
Temporal-only	23,789.1	18.60%	24
Spatial-only	12,609.8	56.85%	0
CAST (ours)	**10,805.0**	**63.03%**	**227**
Capacity ignored	9,958.2	65.93%	20,208

4.5 계산 비용과 확장성

온라인 정책을 주장하려면 한 슬롯을 처리하는 시간이 슬롯 길이보다 충분히 짧아야 한다. 측정은 Apple M1 Max(성능 코어 8, 효율 코어 2)와 메모리 32 GB에서 하였고, 정수 계획은 PuLP 3.3.2에 함께 배포되는 CBC 2.10.3으로 푼다. 최적성 허용오차는 솔버 기본값인 절대 간격 0과 비율 간격 0을 그대로 써서 후보마다 증명된 최적해를 얻는다. 한 슬롯에서 α 후보 11개를 각각 푸는 것이 원칙이지만, 그 슬롯의 대기 작업 수가 모든 리전의 남은 자리보다 적으면 정수 계획의 최적해가 작업마다 가장 싼 리전을 따로 고르는 것과 같아지므로 그때는 풀지 않고 건너뛴다.

슬롯당 시간은 정수 계획을 실제로 푸는지에 따라 갈린다. 건너뛰는 슬롯은 1 ms 아래에서 끝나고 푸는 슬롯은 수백 ms가 든다. 기준 규모에서는 건너뛰는 경우가 1.3%에 그쳐 사실상 매 슬롯을 푸는데, 슬롯당 중앙값 300.9 ms, 평균 296.6 ms, 95 백분위수 339.8 ms, 최악 404.9 ms였다. 슬롯 길이 1시간에 견주면 최악의 경우에도 8,890배의 여유가 있다. 시간 이동의 롤링 재평가는 작업 146,000건 전량이 7.2초에 끝난다. 1년 전체로 외삽하면 공간 이동이 약 43분이다.

표 7 작업 수를 바꾸며 잰 공간 이동의 슬롯당 계산시간. 90일 구간에서 앞 7일을 뺀 슬롯 1,992개이며 유효 상한은 세 규모 모두 12로 고정했다. 시간 이동의 롤링 재평가는 작업 전량에 대해 차례로 2.7초, 7.2초, 17.4초가 걸렸다.

Table 7 Per-slot computation time of spatial shifting across workload sizes, over 1,992 slots of a 90-day window after a 7-day warm-up, with the effective ceiling fixed at 12 in all three. The rolling re-evaluation of temporal shifting took 2.7 s, 7.2 s and 17.4 s over the full job set.

Jobs per slot	Mean (ms)	Max (ms)	ILP solved (%)
8.3 (0.5×)	117.4	441.8	39.5
16.7 (1×)	296.6	404.9	98.7
33.3 (2×)	341.5	427.1	100.0

작업 수를 절반과 두 배로 바꾸어 같은 측정을 반복한 것이 표 7이다. 늘어나는 모양이 선형이 아니다. 절반에서 기준으로 갈 때는 117.4 ms에서 296.6 ms로 2.5배가 되는데, 작업이 두 배가 되어서가 아니라 건너뛸 수 있는 슬롯이 60.5%에서 1.3%로 사라지기 때문이다. 반대로 기준에서 두 배로 갈 때는 296.6 ms에서 341.5 ms로 15%만 는다 — 이미 매 슬롯을 풀고 있어 늘어난 작업이 정수 계획의 크기만 키우기 때문이다. 비용을 정하는 것은 작업 수 자체가 아니라 그 슬롯에서 용량이 묶이는가다. 두 배 규모에서도 최악이 427.1 ms로 슬롯 길이의 8,430분의 1이어서, 작업이 더 늘어도 온라인으로 쓰는 데는 지장이 없다.

5. 결론 및 향후 연구

본 논문은 탄소 인지 스케줄링 시스템 CAST를 제안하였다. 탄소와 지연의 가중치는 매 슬롯 파레토 무릎점에서 자동으로 정한다. 리전별 용량 상한도 매 슬롯 강제하면서 공간 이동과 시간 이동을 결합한다. 여덟 리전의 1년치 실측 탄소집약도 위에서 배출량을 63.03% 줄였고 작업 146,000건 전량을 마감 안에 처리하였다.

본 연구의 주된 발견은 총계가 아니라 분포에 있다. 용량을 두지 않았을 때 시간 이동이 만드는 절감의 71.9%가 캘리포니아 한 리전에서 나오고, 용량 초과가 가장 심한 리전도 캘리포니아다. 상한을 넘긴 시간의 87.4%는 UTC 19~21시, 곧 현지 한낮 세 시간에 몰린다. 반면 종일 평탄한 프랑스는 배정 물량이 가장 많은데도 절감의 2.3%만 냈다. 시간 이동이 값어치를 만드는 리전과 용량이 그 값어치를 가로막는 리전은 같다. 선행 연구가 시간 이동의 예외로 지목한 리전도 캘리포니아다.

이 결과가 실무에 주는 함의는 세 가지다. 첫째, 탄소 인지 스케줄링의 이득을 보고할 때 용량을 함께 보고하지 않으면 그 숫자는 비교할 수 없다. 본 연구보다 낮은 배출을 낸 선행 정책 셋은 모두 같은 상한으로 다시 세면 49,356건 이상을 초과 배정한다. 본 연구의 227건은 마감을 하드 제약으로 둔 결과이며 전체의 0.16%다. 둘째, 시간 이동을 도입할지는 리전별 탄소집약도의 평균이 아니라 변동 폭으로 판단해야 한다(캘리포니아 71.9%, 프랑스 2.3%). 셋째, 예측을 더 잘하는 것보다 용량을 다루는 것이 더 큰 문제다. 예측 오차의 비용은 0.61%포인트다 — 실제 예측(63.03%)과 완전예지 오라클(10,627.3 kg, 63.64%)의 차이다. 반면 용량의 비용은 2.90%포인트로 네 배가 넘는다. 오라클 예측을 넣으면 위반이 227건에서 873건으로 늘어난다 — 예측이 정확할수록 작업이 같은 저탄소 구간에 모이고 그 수렴이 초과를 만든다. 다만 이 배율은 워크로드에 매인다. 등급 분포를 여섯 가지로 바꿔 전 과정을 다시 돌리니 0.23배에서 6.14배까지 벌어졌고, 유연한 작업이 약 27% 아래로 내려가면 순서가 뒤집힌다(본 연구는 39.92%).

한계는 다섯 가지다. 첫째, 작업 발생과 리전 용량을 모두 균등하다고 두었다 — 기준 배출의 69.7%가 인도(24.3%)·일본(16.3%)·한국(15.6%)·텍사스(13.6%)에서 나오는데 실제 수요와 설비는 균등하지 않으므로 분포를 바꾸면 기준값과 절감 폭이 함께 달라진다. 둘째, 실제 배포가 아닌 시뮬레이션이어서 클러스터 관리 계층과의 연동 비용을 확인하지 못했다. 셋째, 캘리포니아·텍사스·독일의 기상 입력이 예보가 아닌 실측값이어서 이 세 리전의 결과는 낙관적일 수 있다. 넷째, 예측 지평을 24시간까지만 시험했다. 다섯째, 지연 등급 분포는 실측이 아니라 설정값이다.

첫째 한계는 조금 더 적어 둔다. 리전 용량을 균등하게 둔 것은 어느 리전도 특별히 넓거나 좁지 않다고 본 것이다. 실제 설비는 리전마다 다르므로, 깨끗한 리전의 용량이 좁으면 본 연구가 보고한 충돌은 더 심해지고 넓으면 덜해진다. 어느 쪽이든 용량을 보지 않을 때 한 리전으로 쏠린다는 관찰 자체는 달라지지 않는다. 비대칭 용량은 따로 실험하지 않았다.

향후 과제는 실제 예보 API 연동, 24시간을 넘는 예측 지평[14], 수요에 비례하는 용량 모델이다. 용량을 지키면서 얻은 절감이 실제 배포에서도 유지되는지 검증할 계획이다.

참고문헌

[1] International Energy Agency, *Energy and AI*, World Energy Outlook Special Report, IEA, Paris, 2025.

[2] European Parliament and Council, Directive (EU) 2023/1791 of the European Parliament and of the Council of 13 September 2023 on Energy Efficiency (recast), *Official Journal of the European Union*, 2023.

[3] A. Radovanović, R. Koningstein, I. Schneider, B. Chen, A. Duarte, B. Roy, D. Xiao, M. Haridasan, P. Hung, N. Care, S. Talukdar, E. Mullen, K. Smith, M. Cottman, and W. Cirne, "Carbon-Aware Computing for Datacenters," *IEEE Transactions on Power Systems*, Vol. 38, No. 2, pp. 1270-1280, Mar. 2023.

[4] A. Souza, S. Jasoria, B. Chakrabarty, D. Irwin, P. Shenoy, A. Bridgwater, A. Lundberg, F. Skogh, and A. Ali-Eldin, "CASPER: Carbon-Aware Scheduling and Provisioning for Distributed Web Services," *Proc. of the 14th Int. Green and Sustainable Computing Conf. (IGSC)*, pp. 67-73, Oct. 2023.

[5] D. Maji, B. Pfaff, V. P R, R. Sreenivasan, V. Firoiu, S. Iyer, C. Josephson, Z. Pan, and R. K. Sitaraman, "Bringing Carbon Awareness to Multi-Cloud Application Delivery," *Proc. of the 2nd Workshop on Sustainable Computer Systems (HotCarbon)*, pp. 1-6, Jul. 2023.

[6] T. Sukprasert, A. Souza, N. Bashir, D. Irwin, and P. Shenoy, "On the Limitations of Carbon-Aware Temporal and Spatial Workload Shifting in the Cloud," *Proc. of the 19th European Conf. on Computer Systems (EuroSys)*, pp. 924-941, Apr. 2024.

[7] W. A. Hanafy, L. Wu, D. Irwin, and P. Shenoy, "CarbonFlex: Enabling Carbon-aware Provisioning and Scheduling for Cloud Clusters," arXiv:2505.18357, 2025.

[8] T. Bahreini, A. N. Tantawi, and O. Tardieu, "Caspian: A Carbon-aware Workload Scheduler in Multi-Cluster Kubernetes Environments," *Proc. of the 32nd IEEE Int. Symp. on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*, pp. 1-8, Oct. 2024.

[9] G. Attenni, Y. Moawad, N. Bartolini, and L. Thamsen, "Spatio-Temporal Shifting to Reduce Carbon, Water, and Land-Use Footprints of Cloud Workloads," arXiv:2512.08725, 2025.

[10] Electricity Maps. API Documentation [Online]. Available: https://docs.electricitymaps.com (downloaded 2026, Oct. 12)
[11] Electricity Maps. Methodology [Online]. Available: https://www.electricitymaps.com/data/methodology (downloaded 2026, Sep. 22)
[12] Microsoft. Azure Network Round-Trip Latency Statistics [Online]. Available: https://learn.microsoft.com/en-us/azure/networking/azure-network-latency (downloaded 2026, Sep. 22)
[13] D. Maji, R. K. Sitaraman, and P. Shenoy, "DACF: Day-Ahead Carbon Intensity Forecasting of Power Grids using Machine Learning," *Proc. of the 13th ACM Int. Conf. on Future Energy Systems (e-Energy)*, pp. 188-192, Jun.-Jul. 2022.
[14] D. Maji, P. Shenoy, and R. K. Sitaraman, "CarbonCast: Multi-Day Forecasting of Grid Carbon Intensity," *Proc. of the 9th ACM Int. Conf. on Systems for Energy-Efficient Buildings, Cities, and Transportation (BuildSys)*, pp. 198-207, Nov. 2022.

부록 A. 선행 정책 재현의 설정 대응

4.4절의 비교는 선행 정책을 개념으로 흉내 낸 것이 아니라, 공개된 구현 코드와 원문 수식을 그대로 옮겨 본 연구와 같은 자료 위에서 실행한 결과다. 모든 재현 구현은 어느 작업을 어느 리전에서 언제 시작하는가만 정하고, 배출량은 구현이 직접 세지 않고 3.5절 식 (15)의 채점 함수 하나를 통과한다. 그 채점 함수가 기준 시나리오를 29,225.6 kg으로 정확히 재현함을 확인한 뒤 사용하였다. 표 8이 정책마다 무엇을 그대로 가져왔고 무엇을 본 연구가 정했는지 가른다.

표 8 선행 정책 재현의 설정 대응. 원문에서 그대로 가져온 것과 본 연구가 정해야 했던 것을 가른다.

Table 8 Setting correspondence for the reproduced prior policies, separating what was taken verbatim from what this work had to set.

Policy	From the original	Set in this work	Why
CASPER	Public code's objective; $\alpha = 0.9$	r_i = jobs in that slot	In the code $\alpha = 0.5$ is the latency baseline
CICS	Eq. (4); no public code	λ_p/λ_e swept 0–5	Ratio not given; result moved 1.0 kg
Sukprasert	Artifact's grid search	Each job's own deadline	Original assumes full knowledge
Caspian	Original IP; LP + greedy	$\rho_j = 1$	No efficiency data; original assumes it

개념으로만 대응시키는 것이 왜 위험한지는 작업 도중 드러났다. 본 연구는 처음에 CASPER의 가중치 α를 탄소와 지연의 절충으로 읽었으나, 원문 표 1은 α를 서버 대수 대비 탄소집약도의 정규화 가중치로 정의한다. 지연은 목적함수가 아니라 별도의 하드 제약이다. 같은 기호가 전혀 다른 것을 절충하고 있었다. 수식을 직접 옮기지 않았다면 이 차이는 표 안으로 조용히 들어갔을 것이다.

원문 파라미터는 본 연구 환경에 맞추어 조이지 않았다. CASPER의 지연 상한 500 ms와 서버당 수용량 10, 리전당 서버 100대를 그대로 두었는데, 본 연구의 리전 간 실측 지연이 최대 244 ms이고 시간당 전체 요청이 최대 38건이므로 두 제약 모두 비활성임이 표본 관측이 아니라 최댓값 비교로 확정된다. 그 결과 CASPER의 해는 매 시각 가장 깨끗한 리전 하나로 전부 보내는 것이 된다. 구현 오류가 아니라 제약이 없는 최적화의 정상적인 귀결이다.

공간과 시간을 함께 쓴 Sukprasert 재현에서는 146,000건의 99.75%가 프랑스로 갔다. 원문도 같은 현상을 보고한다 — 원문 5절은 모든 작업이 스웨덴으로 이동한다고 적고 그 결과를 96% 감축으로 제시한다. 서로 다른 리전 집합과 서로 다른 해의 자료에서 같은 구조가 같은 방식으로 나타난 것이 재현이 옳다는 근거가 된다. 다만 이 배치는 산술로만 성립한다 — 같은 스케줄을 본 연구의 상한으로 다시 세면 120,186건이 위반이다.

본 연구가 스스로에게 부과한 용량 상한은 선행 정책에 부과하지 않았다. 원문에 없는 제약을 임의로 덧씌우면 그것이야말로 유리하게 변형한 비교가 되기 때문이다. 대신 결과를 읽을 때 같은 상한으로 다시 센 위반 건수를 표 6에 함께 적었다. 선행 정책의 낮은 배출이 더 나은 정책에서 온 것이 아니라 더 느슨한 용량 전제에서 온 것임을 그 열이 보인다.

부록 B. 재현 절차

재현은 저장소의 스크립트로 끝난다. 환경은 Python 3.14.7과 PuLP 3.3.2에 함께 배포되는 CBC 2.10.3이며, 측정은 Apple M1 Max(성능 코어 8, 효율 코어 2)와 메모리 32 GB에서 수행하였다. 기준 시나리오 29,225.6 kg과 제안 기법 10,805.0 kg(63.03%, 상한 위반 227건)이 그대로 나와야 한다. 정수 계획 솔버가 동률을 처리하는 방식이 실행마다 달라질 수 있어 총 배출량이 최대 0.14% 흔들리며, 그보다 큰 차이가 나면 입력 자료나 회계가 어긋난 것이다. 작업 명세는 시드 42로 생성기를 돌리면 그대로 다시 만들어지고, 예측 계열은 저장된 평가 기록에서 읽으므로 재현에 모델 추론을 다시 돌릴 필요가 없다.